



Universidade de Brasília
Departamento de Ciência da Computação
Programa de Pós-Graduação em Informática (PPGI)

Arthur Augusto Campos

FLUXO DE CUSTO MÍNIMO EM REDES HÍDRICAS URBANAS

Uma Abordagem via Successive Shortest Paths

Brasília
2026

SUMÁRIO

1	INTRODUÇÃO	3
2	IMPLEMENTAÇÃO	4
2.1	Formulação Matemática	4
2.2	Resolução via Successive Shortest Path (SSP)	5
2.2.1	<i>Condições de Otimalidade</i>	5
2.2.2	<i>Dualidade, Potenciais Nodais e Grafo Residual</i>	5
2.2.3	<i>O Algoritmo SSP e seu Pseudocódigo</i>	6
2.3	Detalhes de Implementação	7
2.4	Instância de Teste	7
2.5	Análise de Complexidade	8
2.6	Resultados Obtidos	9
3	CONCLUSÃO	11
	REFERÊNCIAS	13
	APÊNDICE A - DECLARAÇÃO DO USO DE INTELIGÊNCIA ARTIFICIAL	14

1 INTRODUÇÃO

Para a realização deste trabalho, realizou-se uma análise do transporte ótimo e uma revisão bibliográfica de algoritmos correlatos ao tema. Diante da impossibilidade de abordar esse assunto em sua formulação geral devido à sua grande amplitude, optou-se por focar em uma versão discreta, resgatando a abordagem clássica de seus primórdios.

O problema de transporte ótimo foi introduzido por Gaspard Monge em 1781, motivado pela minimização do custo no deslocamento de terra. Cerca de 150 anos depois, Leonid Kantorovich revisitou essa formulação sob a ótica da programação linear, expandindo as aplicações da teoria, conforme apresentado por Figalli e Glaudo (2021).

Um caso particular dessa linha de estudo é a minimização do custo no envio de fluxos em redes, conhecido como problema de Fluxo de Custo Mínimo (*Minimum Cost Flow* — MCF). Este trabalho se propôs a estudar e aplicar uma solução para uma instância artificial desse modelo.

Como fonte de inspiração para a condução desta pesquisa, realizou-se uma revisão de implementações de algoritmos voltados à resolução do MCF. O artigo norteador do estudo é o desenvolvido por Mascherpa *et al.* (2023), no qual os autores empregam transporte ótimo regularizado para modelar redes de abastecimento hídrico.

Sob a perspectiva dos estudos de Mascherpa *et al.* (2023), a instância artificial proposta neste relatório foi modelada a partir de dados extraídos do Atlas do Distrito Federal (Companhia de Planejamento do Distrito Federal – Codeplan, 2020) e de informações disponíveis no catálogo de metadados geográficos do IPEDF (Instituto de Pesquisa e Estatística do Distrito Federal – IPEDF, 2026).

Para solucionar o MCF na instância em questão, implementou-se em Python o algoritmo **Successive Shortest Paths** (SSP), baseado no pseudocódigo de Ahuja *et al.* (1993).

Por fim, registra-se que este trabalho foi realizado de forma estritamente individual por Arthur Augusto Campos, responsável único por todas as etapas de levantamento teórico, modelagem, desenvolvimento do código-fonte em Python, execução experimental e redação deste relatório.

2 IMPLEMENTAÇÃO

2.1 Formulação Matemática

O problema de alocação hídrica no Distrito Federal foi modelado a partir da formulação matemática formal do problema de Fluxo de Custo Mínimo (MCF), conforme apresentada por Korte e Vygen (2012) em sua definição clássica:

MINIMUM COST FLOW PROBLEM

Instance: A digraph G , capacities $u : E(G) \rightarrow \mathbb{R}_+$, numbers $b : V(G) \rightarrow \mathbb{R}$ with $\sum_{v \in V(G)} b(v) = 0$, and weights $c : E(G) \rightarrow \mathbb{R}$.

Task: Find a b -flow f whose cost $c(f) := \sum_{e \in E(G)} f(e)c(e)$ is minimum (or decide that none exists).

Em termos práticos de programação linear, essa definição formal se traduz no modelo algébrico de otimização a seguir:

$$\min \sum_{(i,j) \in A} c_{ij} x_{ij} \quad (1)$$

sujeito a:

$$\sum_{\{j:(i,j) \in A\}} x_{ij} - \sum_{\{j:(j,i) \in A\}} x_{ji} = b_i \quad \forall i \in V \quad (2)$$

$$0 \leq x_{ij} \leq u_{ij} \quad \forall (i,j) \in A \quad (3)$$

no qual mapeamos os parâmetros físicos da rede de distribuição hídrica da seguinte forma:

- x_{ij} : a vazão de água alocada na adutora (i, j) (fluxo, em l/s);
- b_i : o balanço hídrico do nó i (em l/s). Um valor $b_i > 0$ indica que o nó é uma Estação de Tratamento de Água (ETA — fonte de oferta), enquanto $b_i < 0$ indica uma Região Administrativa (RA — sumidouro de demanda);
- u_{ij} : a capacidade física máxima de vazão da adutora (i, j) (em l/s);
- c_{ij} : a distância geodésica em quilômetros da adutora (i, j) , usada como *proxy* para o custo de bombeamento e transporte.

2.2 Resolução via Successive Shortest Path (SSP)

Como discutido na obra clássica de Ahuja *et al.* (1993), que constitui a obra de referência para a implementação do algoritmo SSP desenvolvida neste trabalho, existem diversos algoritmos fundamentais para a resolução exata do problema de MCF. Um dos métodos mais basilares e intuitivos é o algoritmo *Successive Shortest Path* (SSP), cuja compreensão segue diretamente o fluxo de raciocínio da otimização dual e das condições de otimalidade propostas na referida obra.

2.2.1 Condições de Otimalidade

A certificação de que uma solução viável de fluxo é ótima baseia-se nas condições de custos reduzidos. O resultado central da teoria de fluxos em redes é dado pelo Teorema *Reduced Cost Optimality Conditions*, formalizado na obra de referência da seguinte maneira:

Teorema 2.1 (Condições de Otimalidade por Custo Reduzido). *Uma solução viável x^* é uma solução ótima do problema de fluxo de custo mínimo se, e somente se, para algum conjunto de potenciais nodais π , as seguintes condições de otimalidade por custos reduzidos forem satisfeitas:*

$$\bar{c}_{ij} \geq 0 \quad \forall (i, j) \text{ na rede residual } G(x^*) \quad (4)$$

O leitor interessado na demonstração matemática rigorosa e nas propriedades geométricas desse teorema pode consultar diretamente o capítulo 9 da obra de referência.

2.2.2 Dualidade, Potenciais Nodais e Grafo Residual

A interpretação dessas condições está intimamente ligada à teoria de dualidade em programação linear. Os potenciais de nó π_i atuam como as variáveis duais associadas às restrições de conservação de fluxo em cada nó $i \in V$. Sob a ótica econômica, π_i pode ser interpretado como o preço marginal (ou *shadow price*) da mercadoria (neste caso, a água) no nó i .

Definindo os **custos reduzidos** de cada arco no grafo residual como $\bar{c}_{ij} = c_{ij} - \pi_i + \pi_j$, a condição $\bar{c}_{ij} \geq 0$ estabelece que o custo de transportar uma unidade de fluxo entre os nós i e j deve ser maior ou igual à diferença de preço entre eles. Caso existisse algum arco residual com $\bar{c}_{ij} < 0$, haveria oportunidade de reduzir o custo total enviando fluxo por este caminho (relação com a inexistência de ciclos negativos).

O **grafo residual** G_x é a estrutura que operacionaliza esse conceito dual, representando dinamicamente as direções permitidas para alteração de fluxo:

- **Arcos forward:** representam a capacidade física ainda livre de adução, com capacidade residual $r_{ij} = u_{ij} - x_{ij}$ e custo reduzido $\bar{c}_{ij} = c_{ij} - \pi_i + \pi_j$;
- **Arcos backward:** possibilitam a reversão ou devolução de fluxo previamente enviado, com capacidade residual $r_{ji} = x_{ij}$ e custo reduzido $\bar{c}_{ji} = -c_{ij} - \pi_j + \pi_i$.

2.2.3 O Algoritmo SSP e seu Pseudocódigo

O algoritmo *Successive Shortest Path* tira proveito direto das condições de otimalidade. Diferente de outros métodos que mantêm a viabilidade do fluxo (conservação nos nós) e buscam a otimalidade dual (eliminação de ciclos de custo negativo), o SSP faz o caminho inverso: ele mantém a otimalidade dual (garantindo que $\bar{c}_{ij} \geq 0$ em todas as iterações) e busca a viabilidade primal de forma incremental.

O algoritmo inicia com fluxo nulo e potenciais nodais calculados para satisfazer a otimalidade. A cada iteração, identifica-se nós com excesso de oferta $e(s) > 0$ e nós com déficit de demanda $e(t) < 0$. Em seguida, busca-se o caminho mais curto P de s a t no grafo residual G_x . Como os custos reduzidos são sempre não-negativos, o algoritmo de Dijkstra pode ser empregado com segurança. O fluxo é então aumentado ao longo de P , e os potenciais nodais são atualizados com base nas distâncias calculadas, preservando as condições de otimalidade até que todos os nós atinjam o equilíbrio. O pseudocódigo estruturado do algoritmo é detalhado no algoritmo 1.

Algoritmo 1: Successive Shortest Path

Entrada : Grafo $G = (V, A)$, balanços b , capacidades u , custos c

Saída : Fluxo ótimo x

```

1 Inicializa  $x \leftarrow 0$  (fluxo nulo);
2 Calcula potenciais iniciais  $\pi$  via Bellman-Ford;
3 while existe nó  $s$  com excesso  $e(s) > 0$  do
4   Escolhe nó  $t$  com déficit  $e(t) < 0$ ;
5   Encontra caminho  $P$  de custo mínimo reduzido  $\bar{c}$  de  $s$  a  $t$  em  $G_x$  (Dijkstra
     com  $\bar{c}_{ij} \geq 0$ );
6   Atualiza potenciais  $\pi_i \leftarrow \pi_i - d(i)$  para todo  $i \in V$  (onde  $d(i)$  é a distância
     de  $s$  a  $i$ );
7    $\delta \leftarrow \min(e(s), -e(t), \min_{(i,j) \in P} r_{ij})$ ;
8   Aumenta  $\delta$  unidades de fluxo ao longo de  $P$  no grafo  $x$ ;
9   Atualiza excessos:  $e(s) \leftarrow e(s) - \delta$  e  $e(t) \leftarrow e(t) + \delta$ ;
10 end
11 return  $x$ ;
```

2.3 Detalhes de Implementação

A implementação foi desenvolvida em Python puro, sem dependências de solvers ou bibliotecas externas de otimização. O código é modularizado nos seguintes arquivos:

- `ssp_solucão.py`: script principal que coordena o balanceamento de ponto flutuante, carrega os dados e executa o solucionador SSP, além de gerar relatórios de saída;
- `ssp_algorithm.py`: loop principal do algoritmo SSP, estruturado para busca sucessiva de fontes de excesso e sumidouros de déficit;
- `grafo.py`: definição da estrutura e dados da instância de teste (nós, balanços, adutoras, capacidades e custos) e geração de visualizações gráficas;
- `tools.py`: funções auxiliares modulares contendo Bellman-Ford para potenciais iniciais, construção do grafo residual, busca de caminho mínimo via algoritmo de Dijkstra (utilizando busca linear) e reconstrução do caminho.

Principais decisões de implementação:

- Dijkstra com busca linear simples (sem heap binário) para manter a implementação didática, resultando em $O(V^2)$ por iteração;
- Grafo residual reconstruído a cada iteração como lista de adjacência dinâmica;
- Tolerância numérica de 10^{-5} para comparações de fluxo;
- Verificação automática de otimalidade ao final via custos reduzidos $\bar{c}_{ij} \geq 0$.

2.4 Instância de Teste

A instância utilizada é uma simplificação da rede hídrica real do Distrito Federal, com dados baseados em estatísticas da CAESB e ADASA:

- **Topologia**: grafo bipartido com 7 ETAs (fontes) e 7 RAs (sumidouros), interconectados por 26 adutoras;
- **Injeção total**: +7.600 l/s distribuídos entre as ETAs;
- **Demanda total**: -7.600 l/s distribuídos entre as RAs;
- **Custos**: distância geodésica em km entre ETA e RA (calculada via fórmula de Haversine a partir de coordenadas reais).

Nó	Tipo	Balanço Nodal (l/s)
ETA Rio Descoberto	Fonte	+4450,00
ETA Brasília	Fonte	+1200,00
ETA Lago Norte	Fonte	+700,00
ETA Sobradinho	Fonte	+570,00
ETA Gama	Fonte	+320,00
ETA Paranoá	Fonte	+300,00
ETA Planaltina	Fonte	+60,00
Plano Piloto	Sumidouro	-1763,40
Sobradinho	Sumidouro	-1101,09
Guará	Sumidouro	-1067,97
Taguatinga	Sumidouro	-1051,42
Planaltina	Sumidouro	-927,23
Samambaia	Sumidouro	-852,72
Ceilândia	Sumidouro	-836,17

Tabela 1 - Balanços nodais da instância hídrica do DF.

2.5 Análise de Complexidade

O custo computacional total do algoritmo Successive Shortest Path (SSP) é dominado pela execução inicial do algoritmo de Bellman-Ford e pelas chamadas sucessivas do algoritmo de Dijkstra para a determinação de caminhos mínimos de custo reduzido no grafo residual. A tabela 2 detalha a complexidade de cada etapa do algoritmo.

Etapa	Operação	Estrutura	Complexidade
Inicialização	Fluxos $x = 0$	Vetor	$O(E)$
Potenciais iniciais	Bellman-Ford multi-source	Arestas	$O(V \cdot E)$
Busca de fontes/sumidouros	Varredura linear	Vetor	$O(V)$
Caminho mínimo residual	Dijkstra (lista linear)	Lista/Fila	$O(V^2)$
Atualização de potenciais	Vetor de distâncias	Vetor	$O(V)$
Aumento de fluxo	Busca reversa (ponteiros de pai)	Caminho	$O(V)$

Tabela 2 - Complexidade por etapa do SSP.

Analisando as etapas dominantes, temos:

1. **Pré-processamento (Bellman-Ford):** Utilizado uma única vez para obter os potenciais iniciais π e garantir custos reduzidos residuais não-negativos ($\bar{c}_{ij} \geq 0$). Essa etapa consome $O(V \cdot E)$ operações.

2. **Loop Principal (Dijkstra):** Em cada iteração, busca-se um caminho mínimo no grafo residual. Como o Dijkstra foi implementado utilizando uma busca linear simples para extração do nó de menor distância na fila de prioridades (sem o uso de heap binário), a complexidade por iteração é de $O(V^2)$.
3. **Limite Superior da Complexidade (Notação Big-O):** O custo total de execução é limitado superiormente por $O(V \cdot E + U \cdot V^2)$, em que $U = \sum_{b_i > 0} b_i$ representa a soma dos excessos das fontes. Este limite é estabelecido para instâncias com balanços e capacidades inteiros. Para a instância real utilizada, com capacidades e demandas fracionárias, o algoritmo convergiu em apenas 14 iterações.

2.6 Resultados Obtidos

O algoritmo SSP convergiu em **14 iterações** e encontrou a solução ótima com custo total de **129.482,24 km-l/s**. A verificação automática de otimalidade confirmou que todos os custos reduzidos no grafo residual são não-negativos.

Origem	Destino	Fluxo (l/s)	Utilização (%)
ETA Rio Descoberto	Ceilândia	836,17	13,9%
ETA Rio Descoberto	Samambaia	532,72	8,9%
ETA Rio Descoberto	Taguatinga	1051,42	17,5%
ETA Rio Descoberto	Guará	1067,97	53,4%
ETA Rio Descoberto	Plano Piloto	961,72	48,1%
ETA Brasília	Plano Piloto	801,68	66,8%
ETA Brasília	Sobradinho	398,32	39,8%
ETA Gama	Samambaia	320,00	100,0%
ETA Lago Norte	Planaltina	700,00	70,0%
ETA Paranoá	Sobradinho	300,00	100,0%
ETA Planaltina	Planaltina	60,00	6,0%
ETA Sobradinho	Sobradinho	402,77	67,1%
ETA Sobradinho	Planaltina	167,23	16,7%

Tabela 3 - Alocação ótima de fluxos nas adutoras.

Observações sobre os resultados:

- **Arcos saturados** (100%): Gama → Samambaia e Paranoá → Sobradinho — indicam gargalos da rede;
- **Arcos ociosos**: 13 adutoras operam com fluxo zero, indicando rotas menos eficientes;

- **Tempo de execução:** inferior a 1 ms para a instância de 14 nós e 26 arestas;
- A iteração 8 demonstra o uso de **arcos reversos** no grafo residual (caminho: ETA Paranoá → Sobradinho → ETA Sobradinho → Planaltina), validando a capacidade do algoritmo de autocorreção.

3 CONCLUSÃO

O presente trabalho demonstrou a aplicação do algoritmo Successive Shortest Path (SSP) ao problema de Fluxo de Custo Mínimo em uma rede hídrica simplificada do Distrito Federal. O algoritmo convergiu em 14 iterações, encontrando a alocação ótima com custo total de 129.482,24 km·l/s, e a verificação automática de otimalidade via custos reduzidos confirmou a corretude da solução.

A formulação matemática do problema e a construção do grafo residual com arcos reversos foram fundamentais para modelar o cenário, permitindo que o algoritmo revesse fluxos já estabelecidos nas iterações anteriores e garantindo a otimalidade global ao final do processo.

As principais dificuldades encontradas durante o desenvolvimento foram:

- **Delimitação do escopo:** a dificuldade de selecionar um tópico de transporte ótimo específico que coubesse no escopo da disciplina e no tempo pré-definido para a realização do estudo;
- **Limitação de dados:** a necessidade de consolidar e realizar a modelagem matemática da rede utilizando exclusivamente dados públicos e abertos disponíveis na internet (provenientes do IPEDF, CAESB e ADASA);
- **Balanceamento nodal:** a instância real apresentou pequenas inconsistências entre oferta e demanda total, exigindo um ajuste de absorção de resíduo numérico para garantir a viabilidade do fluxo;
- **Precisão numérica:** operações com ponto flutuante requerem tolerâncias numéricas adequadas (10^{-5}) para evitar falsos positivos nas verificações de otimalidade e balanço;
- **Grafo residual:** a compreensão teórica do funcionamento e do papel dinâmico do grafo residual (composto por caminhos de aumento e arcos reversos), o qual é essencial para garantir a otimalidade global.

Por fim, como limitação e sugestão de melhoria para o trabalho, uma comparação sistemática com outros métodos clássicos propostos por Ahuja *et al.* (1993) para a resolução do MCF enriqueceria a análise do algoritmo. Especificamente, teriam sido valiosas as implementações comparativas com os algoritmos de Cancelamento de Ciclos (*Cycle-Canceling*), Primal-Dual, Out-of-Kilter e de Relaxamento (*Relaxation Algorithm*), permitindo avaliar o desempenho relativo sob diferentes topologias de rede.

Como trabalho futuro, a inserção de incertezas nas demandas — modelando-as como distribuições de probabilidade — transformaria o problema determinístico em um Problema de Ponte de Schrödinger. Isso conectaria o modelo proposto à fronteira atual

de pesquisa em transporte ótimo regularizado, em linha com o estado da arte sugerido por Mascherpa *et al.* (2023).

REFERÊNCIAS

AHUJA, Ravindra K. *et al.* **Network Flows: Theory, Algorithms, and Applications**. [S. l.]: Prentice Hall, 1993.

COMPANHIA DE PLANEJAMENTO DO DISTRITO FEDERAL – CODEPLAN. **Atlas do Distrito Federal 2020: Infraestrutura**. Brasília, 2020.

CORMEN, Thomas H. *et al.* **Introduction to Algorithms**. 3rd. [S. l.]: MIT Press, 2009.

CUTURI, Marco. Sinkhorn Distances: Lightspeed Computation of Optimal Transport. **Advances in Neural Information Processing Systems (NeurIPS)**, v. 26, p. 2292–2300, 2013.

FIGALLI, Alessio; GLAUDO, Federico. **An Invitation to Optimal Transport, Wasserstein Distances, and Gradient Flows**. [S. l.]: EMS Press, 2021.

INSTITUTO DE PESQUISA E ESTATÍSTICA DO DISTRITO FEDERAL – IPEDF. **Catálogo de Dados Geográficos do Distrito Federal**. 2026. Disponível em: <https://catalogo.ipe.df.gov.br>. Acesso em: 16 jun. 2026.

KORTE, Bernhard; VYGEN, Jens. **Combinatorial Optimization: Theory and Algorithms**. 5th. [S. l.]: Springer, 2012.

MASCHERPA, Michele *et al.* Estimating pollution spread in water networks as a Schrödinger bridge problem with partial information. **European Journal of Control**, Elsevier, v. 74, p. 100846, 2023.

RUBNER, Yossi *et al.* The Earth Mover's Distance as a Metric for Image Retrieval. **International Journal of Computer Vision**, Springer, v. 40, n. 2, p. 99–121, 2000.

APÊNDICE A - DECLARAÇÃO DO USO DE INTELIGÊNCIA ARTIFICIAL

Declaro, para os devidos fins, que o desenvolvimento deste trabalho contou com o suporte de ferramentas baseadas em Inteligência Artificial Generativa. O uso desses recursos ocorreu de forma assistida e complementar, com o propósito de acelerar e otimizar as etapas de pesquisa, desenvolvimento e redação.

O emprego das tecnologias distribuiu-se da seguinte forma:

- **Claude:** Utilizado para a compreensão conceitual e estudos preliminares sobre o problema de transporte ótimo, além de auxiliar no processo de *brainstorming* para a delimitação de escopo e restrições da modelagem;
- **Gemini:** Empregado como suporte na revisão bibliográfica e catalogação das fontes bibliográficas iniciais;
- **Assistente Antigravity (utilizando os modelos Gemini 3.5 Flash e Claude 4.6 Opus):** Utilizado majoritariamente na escrita deste relatório, elaboração dos slides de apresentação, codificação da solução em Python e elucidação de tópicos específicos com base na bibliografia previamente selecionada.

Ressalta-se que todos os códigos, textos, equações e análises produzidos com o suporte dessas ferramentas foram revisados, validados e submetidos a um criterioso processo de avaliação e curadoria humana conduzido integralmente pelo autor. Desse modo, o recurso à Inteligência Artificial serviu estritamente como agente facilitador de produtividade científica, preservando a autoria, a responsabilidade acadêmica e o rigor conceitual do trabalho apresentado.

Softwares e modelos utilizados: Claude (Anthropic), Gemini (Google) e assistente Antigravity (com modelos Gemini 3.5 Flash e Claude 4.6 Opus).